CRAZY AI & ROBOTICS

Graphene Materials Transformer V5

CrystalNN + Geometric-Bias Transformer + Multi-task Learning
for Graphene-like Materials Screening

Mini Project Presentation · 10–12 min · Materials Project + PyTorch

발표 시간: 10분

WHY GRAPHENE-LIKE MATERIALS?

Unique Properties

High carrier mobility
Dirac cone / linear dispersion
Mechanical strength
Thermal conductivity
→ Electronics, sensors, composites

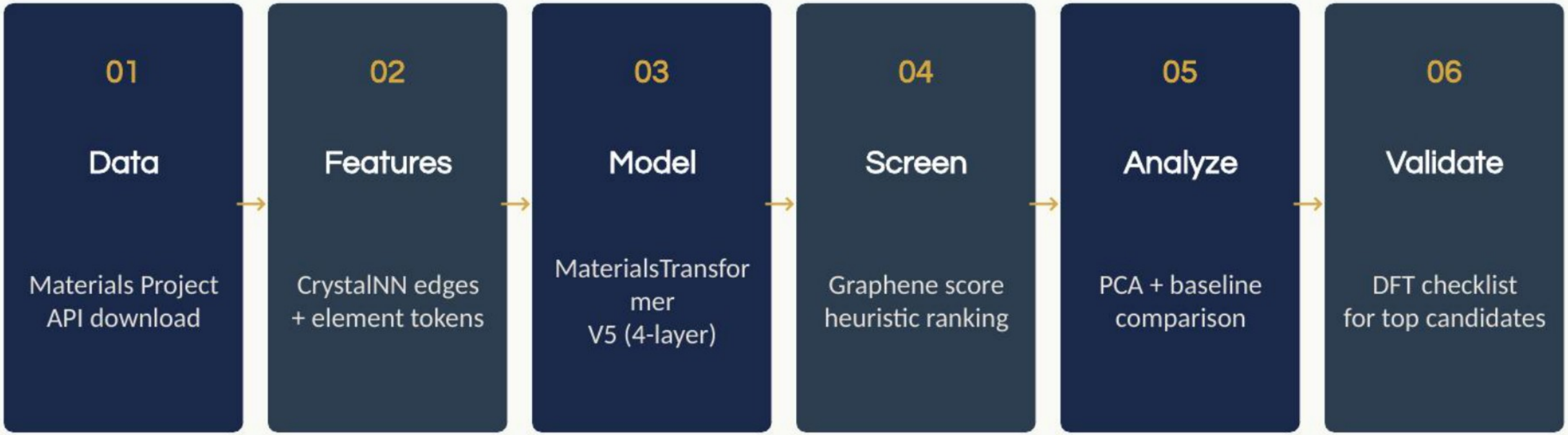
Discovery Bottleneck

Experimental synthesis is slow
DFT (VASP etc.) is expensive
Millions of candidates in MP
Need fast, structure-aware filter
before expensive validation

Our Goal

Learn formation energy,
band gap, energy above hull
from crystal structure
+ structural proxies for
sp^2 / 3-coordinate character
→ Rank graphene-like candidates

END-TO-END PIPELINE



Key principle: Transformer predicts thermodynamic & electronic properties; graphene_score is a structural heuristic only — not a probability or DFT result.

All rankings require independent DFT / experimental verification.

MaterialsTransformerV5 ARCHITECTURE

Input	Atomic number sequence + 9-dim elemental features
Optional Edge	Bond-length edge_attr → Geometric Attention Bias
Encoder	4-layer Pre-LN Transformer (interaction learning)
Pooling	Attention Pooling → fixed-size material vector
Heads	3 regression heads: E_form / Band gap / E_hull

V5 DESIGN CHOICES

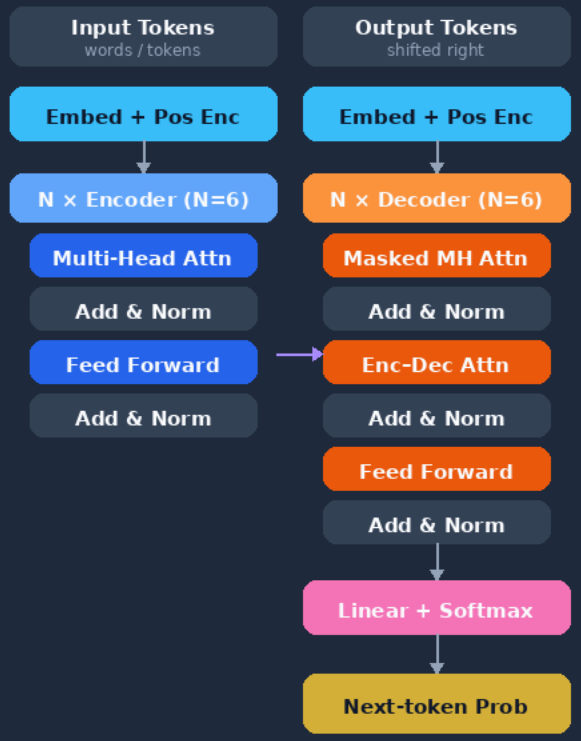
- Pre-LN for stable training
- Geometric Bias injects bond-length geometry into every attention head
- Multi-task learning shares representation across three property targets
- Attention pooling handles variable-length structures
- Train / Val / Test split with MAE · RMSE · R^2

ARCHITECTURE COMPARISON

Attention Is All You Need (2017) vs CRAZY AI Materials Transformer V5.3-C

Original Transformer (2017)

Encoder–Decoder | Seq2Seq



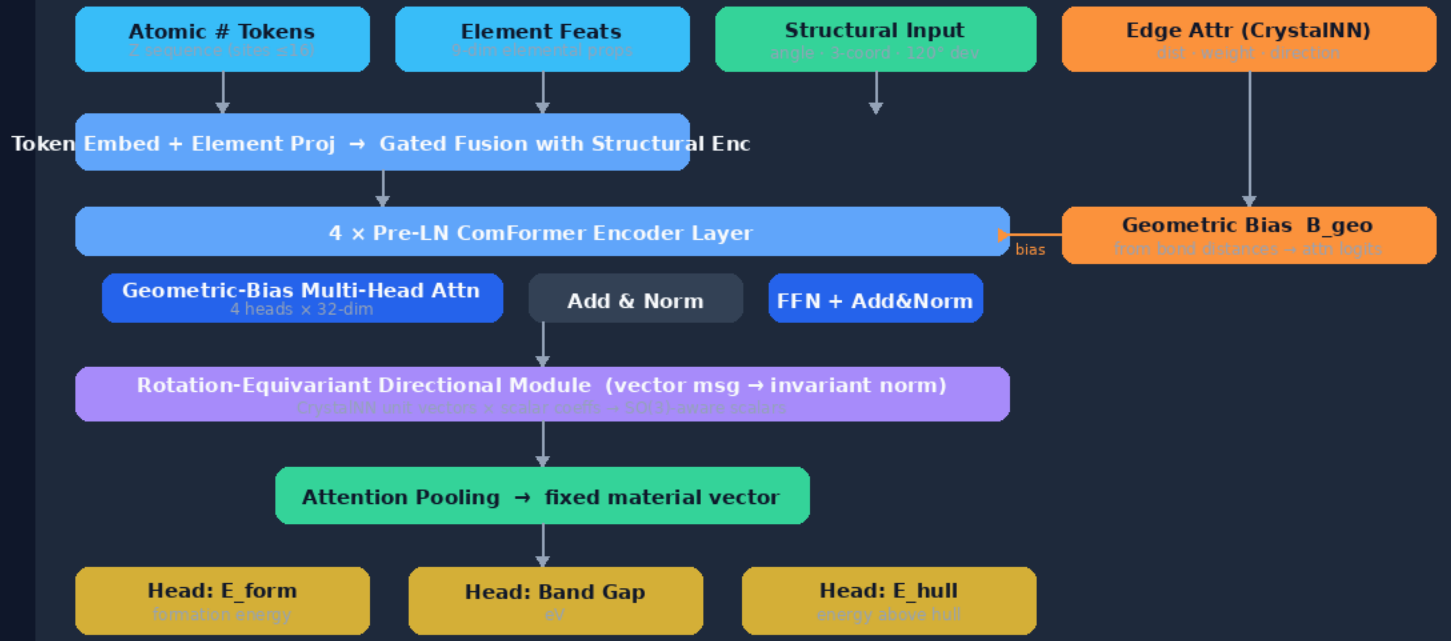
Key traits

- Full encoder–decoder stack
- Masked self-attn for autoregression
- Cross-attention links Enc→Dec
- d_model=512, 8 heads, 6 layers
- Goal: sequence generation

VS

Materials Transformer V5.3-C (Modified)

Encoder-only | Multi-task Regression | Geometric Bias



V5 Modifications vs Original

1. Decoder removed — no autoregressive generation; pure property regression
2. Geometric Bias — bond-length edge features added to every attention logit
3. Structural Input Encoder — mean bond angle, C 3-coord ratio, 120° MAE gated into tokens
4. Directional Module — CrystalNN direction vectors → rotation-invariant norms
5. Compact size — d_model=128, 4 layers, 4 heads (vs 512 / 6 / 8)
6. Multi-task heads — shared encoder, three independent linear regression heads
7. Attention Pooling — variable-length atom sequence → fixed material embedding

TRANSFORMER CORE — Geometric Bias Attention

Data Flow

Atoms (Z) + Element Feats

Embedding + Linear → d_model

× 4 × ComFormerEncoderLayer

Pre-LN → GeoBiasAttn → FFN

Attention Pooling

atom tokens → material vector

3 Heads

E_form · BandGap · E_hull

Key classes (simplified)

```
class GeometricBiasMultiheadAttention:
  # scores = QKᵀ / √dₖ  +  geo_bias
  # geo_bias ← edge_attr (bond length)

class ComFormerEncoderLayer:
  x = x + Attn( LN(x), edges )
  x = x + FFN( LN(x) )

class MaterialsTransformerV5:
  emb = [AtomicEmbed(Z) || FeatProj]
  for layer in layers:  # depth=4
      emb = layer(emb, mask, edges)
  h = AttentionPooling(emb)
  → heads[formation / gap / ehull]
```

CrystalNN — Local Environment → Edges for Geometric Bias

What CrystalNN provides

Neighbor list

Per-atom local neighbors
(not experimental bonds)

edge_index

(2, E) source / dest
atom indices

edge_attr

Bond length (Å)
→ Geometric Bias input

Proxies

3-coord fraction
angle / C–C deviation

$$\text{Attention Score} = QK^T/\sqrt{d_k} + \text{Geometric_Bias(bond length)}$$

CrystalNN is a structural analysis tool. It does not prove chemical bonds or sp^2 hybridization.

extract_structure_and_edges (core)

```
from pymatgen.analysis.local_env
    import CrystalNN

cnn = CrystalNN()
for i in range(n_sites):
  nbs = cnn.get_nn_info(struct, i)
  if len(nbs) == 3:              # 3-coord
      three_coord += 1
  for nb in nbs:
      j = nb['site_index']
      d = struct.get_distance(i, j)
      edge_src.append(i)
      edge_dst.append(j)
      edge_len.append(d)     # Å

# → edge_index, edge_attr
# → 3-coord / angle / C–C proxies
```

STRUCTURAL PROXIES FOR GRAPHENE-LIKENESS

CrystalNN Features

Local neighborhood estimator
(not experimental bond proof)

- 3-coordinate fraction (overall & C)
- C–C bond length deviation from 1.42 Å
- sp^2 angle deviation (target ~120°)
- Mean coordination number
- Carbon / Group-14 ratio
- Estimated dimensionality

graphene_score

Heuristic ranking only

Combines structural proxies
with predicted properties
to prioritize candidates
for further study.

⚠️ Does NOT equal:
- Probability of being graphene
- Proof of sp^2 hybridization
- DFT or experimental result

```
# ============================================================
# GeometricBiasMultiheadAttention
# 결합 길이(거리) 정보를 RBF로 펼친 뒤, 헤드별 기하 편향으로
# Self-Attention 점수에 더해 주는 모듈
# ============================================================
class GeometricBiasMultiheadAttention(nn.Module):
    def __init__(self, embed_dim, num_heads, rbf_dim=32):
        # embed_dim : 토큰 임베딩 차원 (예: 128)
# num_heads : 멀티헤드 개수 (예: 4)
# rbf_dim   : 결합 길이 RBF 전개 차원 (예: 32)
        super().__init__()
        # 표준 MultiheadAttention (batch_first=True → [B,N,D] 입력)
        self.mha = nn.MultiheadAttention(
            embed_dim, num_heads, batch_first=True)
        # RBF 특성 → 헤드 수만큼의 스칼라 편향으로 사영
# 출력 shape 예: [B, N, N, num_heads]
        self.rbf_proj = nn.Linear(rbf_dim, num_heads)

    def forward(self, x, bond_dists=None, key_padding_mask=None):
        # x          : [B, N, D] 원자(사이트) 토큰 표현
# bond_dists : [B, N, N] 사이트 간 결합 길이 (거리 행렬)
# key_padding_mask : 패딩 위치 마스크
# ------------------------------------------------------
# Attention Score = (QK^T)/√d_k + Geometric_Bias(bond_dists)
# ------------------------------------------------------
if bond_dists is not None:
            # 1) 거리를 RBF 가우시안 전개 → 연속 기하 특성
            rbf_feats = rbf_expansion(bond_dists)  # [B,N,N,rbf_dim]
# 2) 헤드별 편향으로 선형 사영 후 축 재배치
            bias = self.rbf_proj(rbf_feats).permute(0, 3, 1, 2)
            # bias: [B, num_heads, N, N] → MHA attn_mask로 사용
else:
            bias = None  # 거리 정보 없으면 기하 편향 생략
# 3) 표준 MHA 호출. attn_mask에 기하 편향 주입
        out, _ = self.mha(
            x, x, x,
```

Analysis / 심층 분석

- **RBF expansion** of bond distances → continuous geometry features
- 결합 길이 RBF 전개 → 연속 기하 특성으로 변환
- **rbf_proj**: RBF dims → per-head bias (head-specific geometry)
- 헤드별로 다른 기하 편향을 학습
- Injected via `attn_mask` of standard MHA
- 표준 MultiheadAttention의 attn_mask 경로 사용 → 구현 단순
- Physical prior: closer atoms get stronger attention
- 물리적 prior — 가까운 원자에 더 강한 attention
- RBF is smooth & translation-invariant vs raw MLP bias
- MLP 직접 bias 대비 RBF가 더 smooth하고 불변

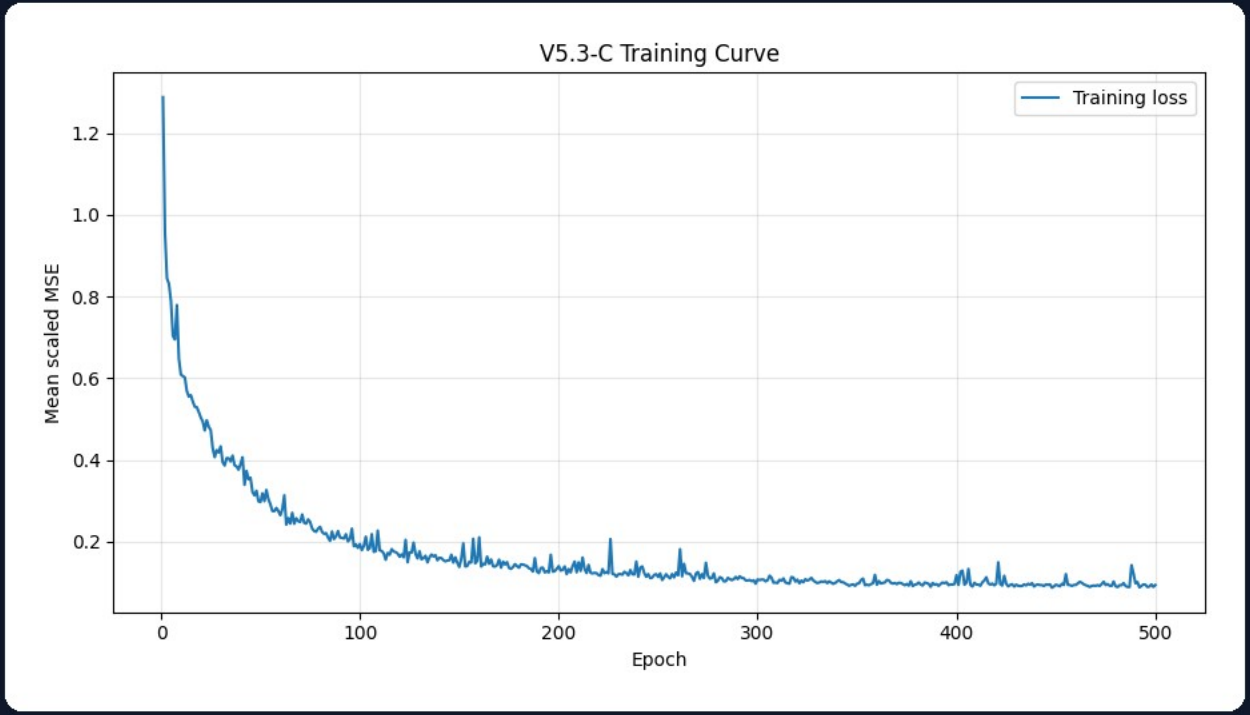
```python
# ============================================================
# MaterialsTransformerV5
# 원자번호·원소특성 임베딩 → ComFormer 인코더 스택 →
# Attention Pooling → 3개 멀티태스크 회귀 헤드
# (형성에너지 / 밴드갭 / Convex Hull 위 에너지)
# ============================================================
class MaterialsTransformerV5(nn.Module):
    def __init__(self, num_species=95, elem_dim=9,
                 embed_dim=128, depth=4):
        # num_species : 지원 원자 종류 수 (원자번호 임베딩 테이블 크기)
# elem_dim    : 원소 특성 벡터 차원 (예: 주기·족·반경 등 9차원)
# embed_dim   : 모델 숨김 차원 d_model (예: 128)
# depth       : ComFormer 인코더 층 수 (예: 4)
        super().__init__()
        # 원자번호(Z) → 학습 가능한 임베딩 벡터
        self.atom_embed = nn.Embedding(num_species, embed_dim)
        # 연속 원소 특성 → embed_dim 으로 선형 투영
        self.elem_proj  = nn.Linear(elem_dim, embed_dim)
        # GeometricBias MHA 를 포함한 Pre-LN 인코더 블록 × depth
        self.layers = nn.ModuleList([
            ComFormerEncoderLayer(embed_dim, num_heads=4)
            for _ in range(depth)
        ])
        # 사이트 토큰들을 가중 합산하는 학습 가능 풀링
        self.pool = AttentionPooling(embed_dim)
        # ----- 3 Multi-Task Heads (풀링 후 독립 선형층) -----
        self.head_ef    = nn.Linear(embed_dim, 1)  # Formation Energy (eV/atom)
        self.head_eg    = nn.Linear(embed_dim, 1)  # Band Gap (eV)
        self.head_ehull = nn.Linear(embed_dim, 1)  # Energy Above Hull (eV/atom)
```

Analysis / 심층 분석

- **ComFormerEncoderLayer**: Pre-LN + GeometricBias MHA
- 조성 인식 Pre-LN 블록 + GeometricBias MHA
- Atom embed (Z) + element feature projection → fused token
- 원자번호 임베딩 + 원소 특성 투영 결합
- **AttentionPooling**: learnable site weights (mask-aware)
- 사이트 가중 합산 (패딩 마스크 반영)
- 3 independent heads after pool → multi-task
- 풀링 후 독립 헤드로 타깃별 스케일 대응
- depth=4, d=128: Colab T4 memory trade-off
- Colab GPU 메모리와 모델 용량의 균형

TRAINING DYNAMICS — V5.3-C

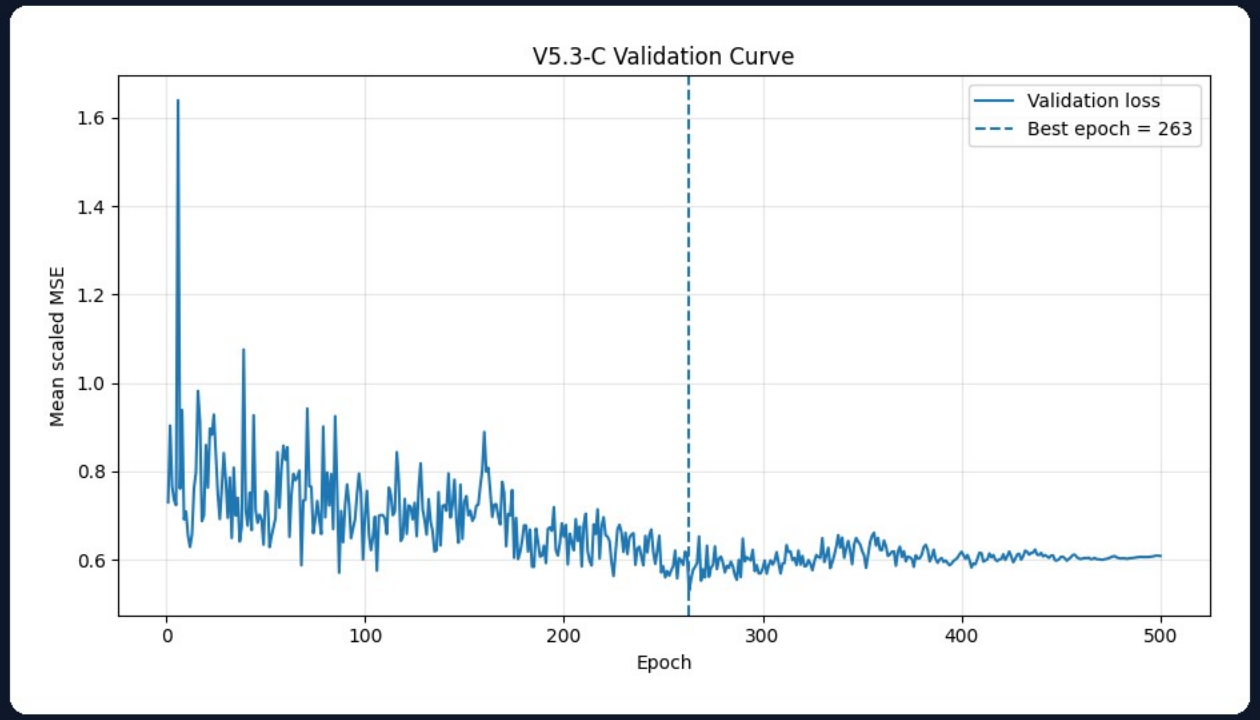
Train loss over 500 epochs (best val checkpoint at epoch 263)



Scaled MSE on train set · CosineAnnealingLR · AdamW

VALIDATION CURVE — V5.3-C

Validation loss; dashed line marks best checkpoint (epoch 263)



Best checkpoint selected by validation loss only · test set never used for selection

TOP-RANKED GRAPHENE-LIKE CANDIDATES

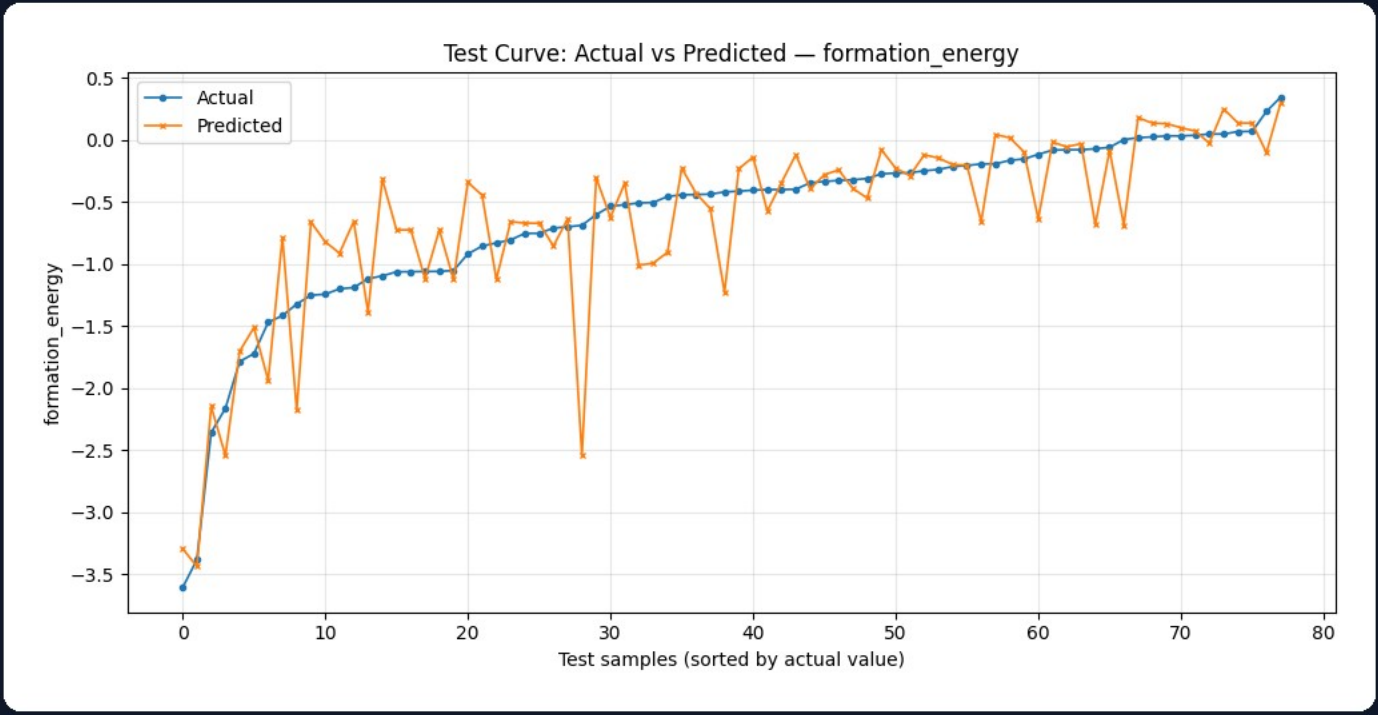
Rank	mp-id	Formula	Score	Key Notes
1	mp-4691	Ag_2CO_3	4.24	Highest score; C 3-coord present
2	mp-1029975	$Al_2(CN_2)_3$	2.29	CN_2 units; interesting C environment
3	mp-561620	Ag_3AsS_3	2.25	High 3-coord fraction (non-C)
4	mp-552185	$AgNO_3$	2.05	Layered-related structure cues
5	mp-9956	$AlCr_2C$	2.04	Carbide; group-14 context

Note: Scores are heuristic rankings. All candidates require DFT (band structure, phonon, 2D stability) before any claim of graphene-like behavior.

Example: $Al_2(CN_2)_3$ is highlighted in the notebook as a case where AI prediction must be separated from DFT validation.

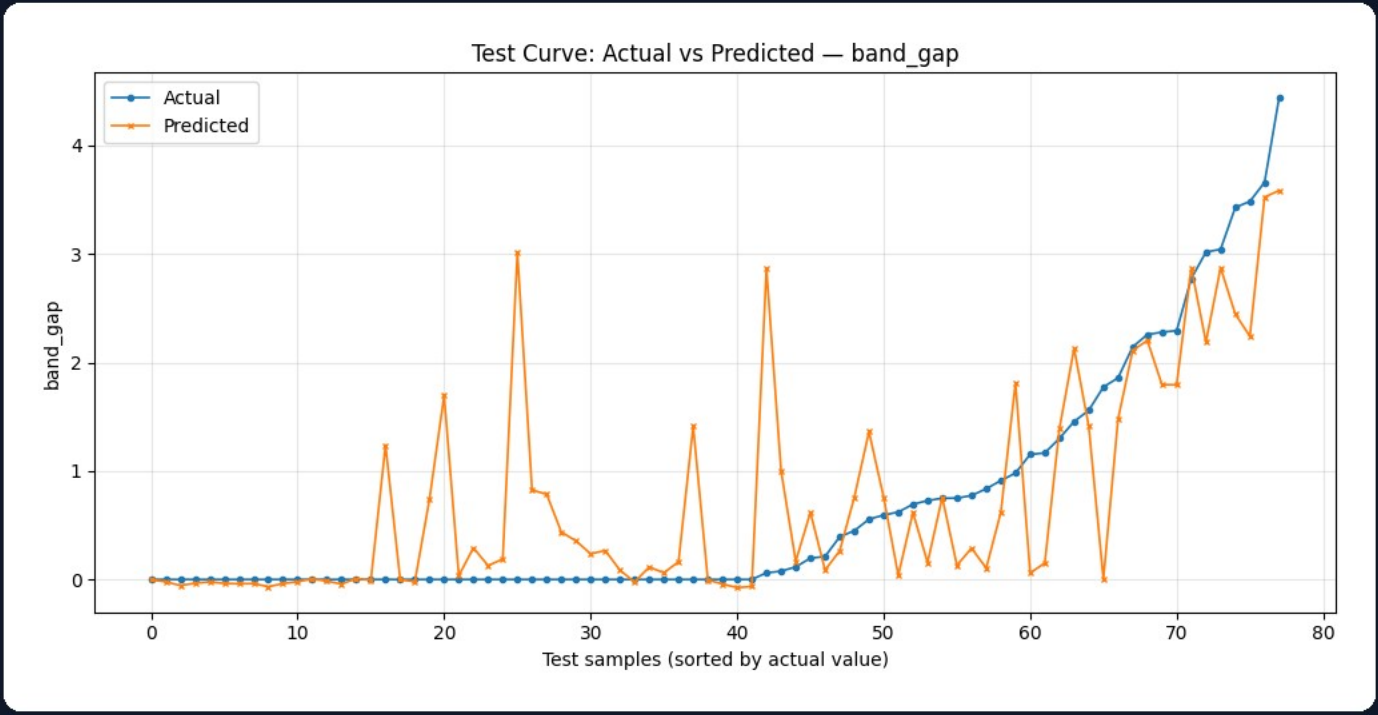
TEST: FORMATION ENERGY — Actual vs Predicted

Held-out test set sorted by actual value (formula-group split)



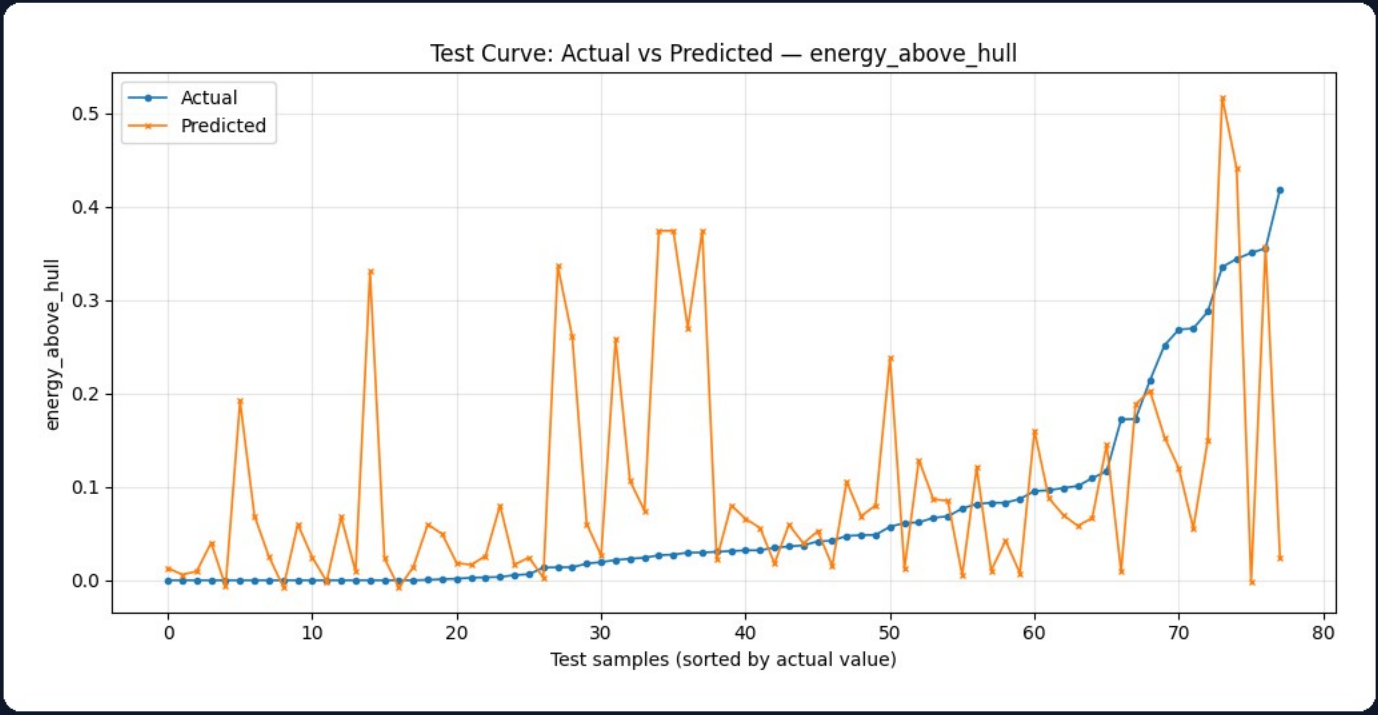
TEST: BAND GAP — Actual vs Predicted

Held-out test set sorted by actual value (formula-group split)



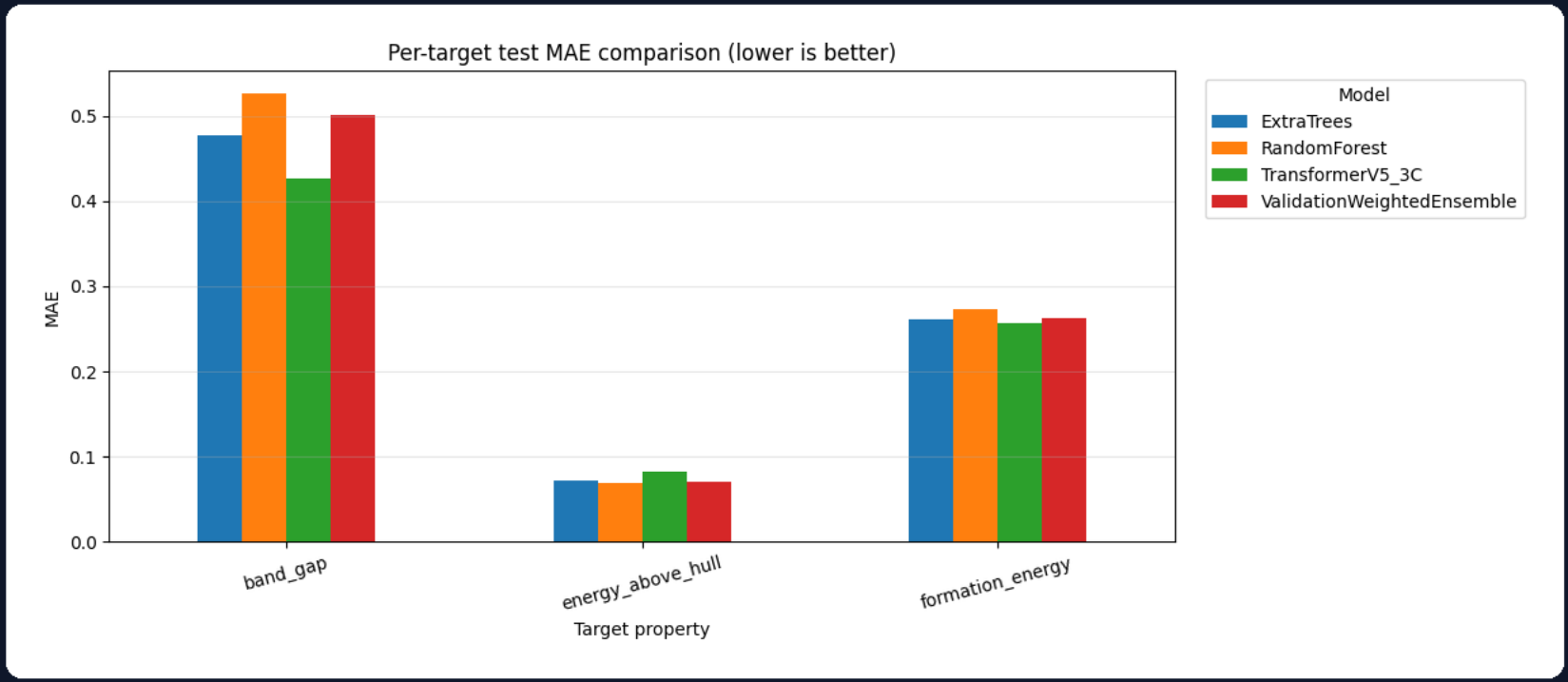
TEST: ENERGY ABOVE HULL — Actual vs Predicted

Held-out test set sorted by actual value (formula-group split)



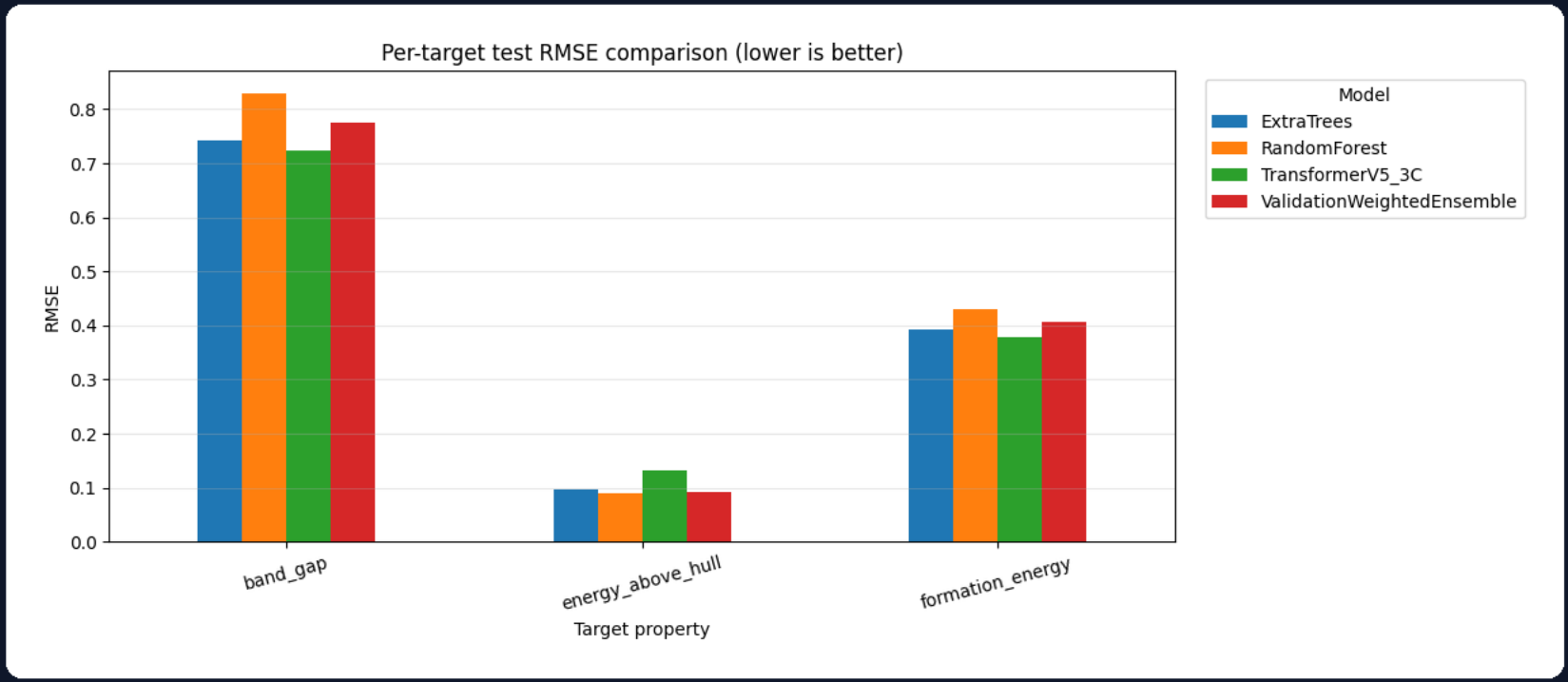
MODEL COMPARISON — Per-target Test MAE

Lower is better · TransformerV5_3C vs ExtraTrees / RandomForest / Ensemble



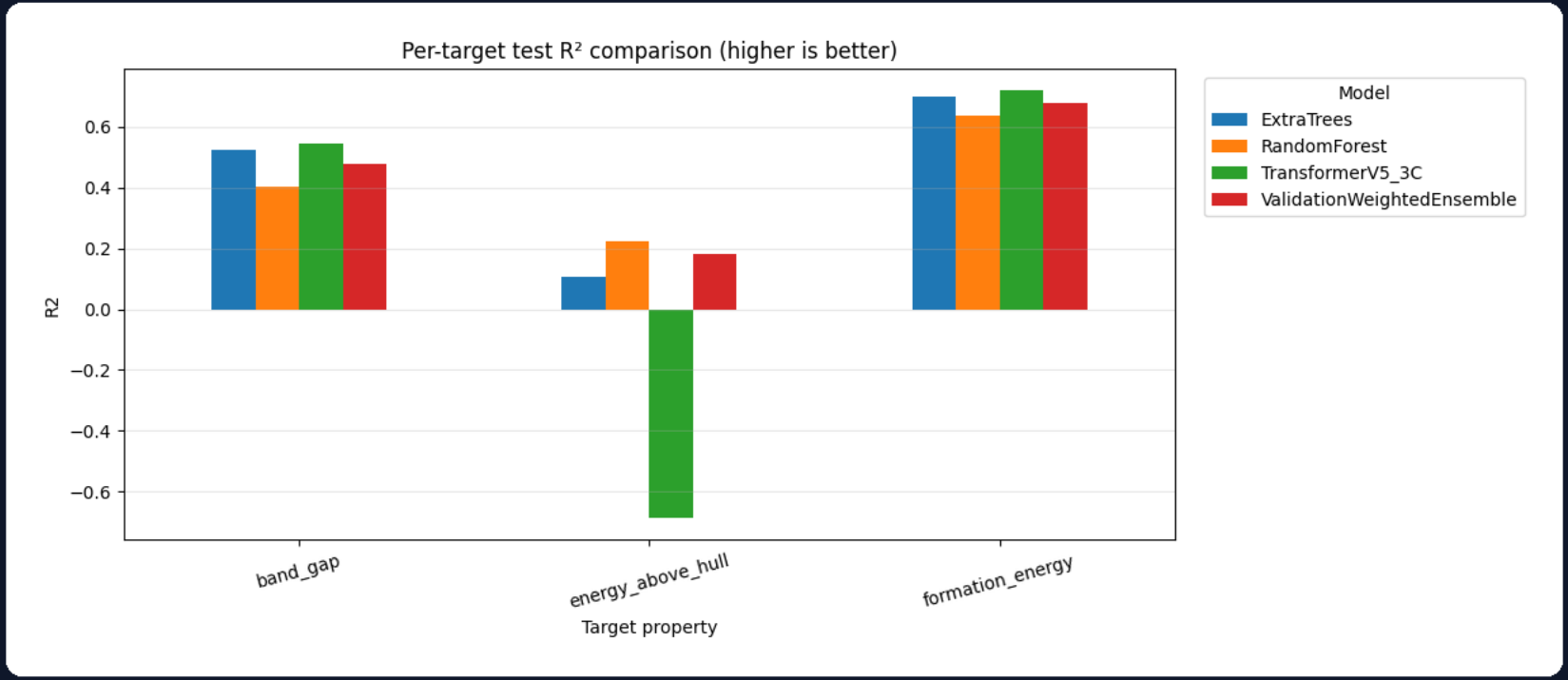
MODEL COMPARISON — Per-target Test RMSE

Lower is better · Independent baselines on the same held-out test set



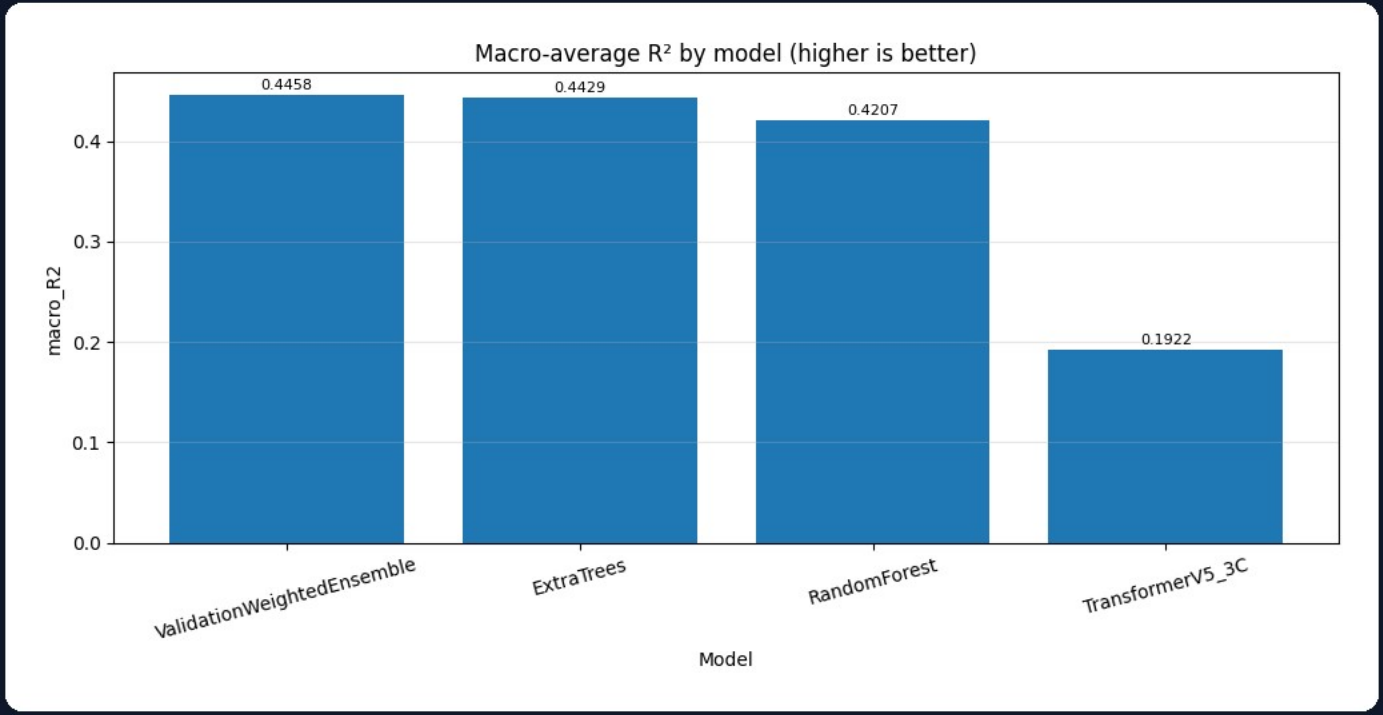
MODEL COMPARISON — Per-target Test R²

Higher is better · Note: Transformer R² on energy_above_hull is negative (hard target)



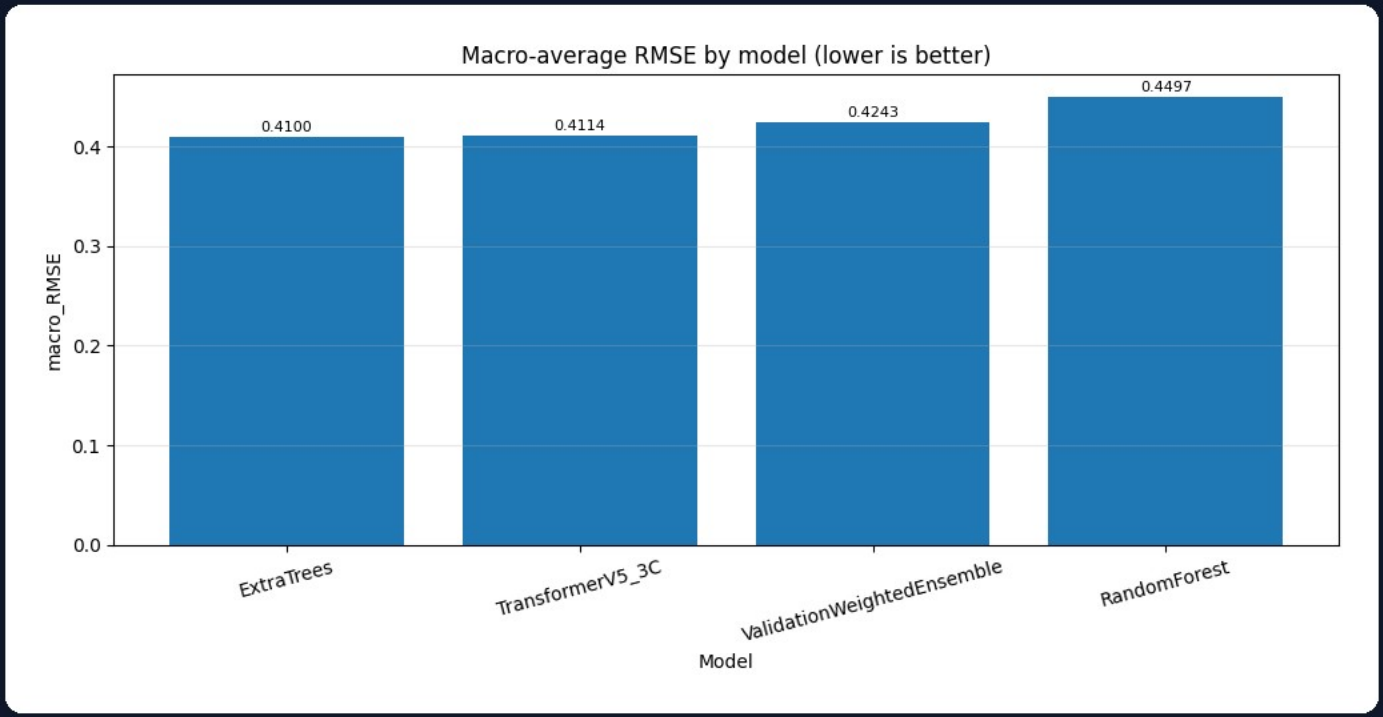
MACRO-AVERAGE R² BY MODEL

Overall ranking of models across the three targets



MACRO-AVERAGE RMSE BY MODEL

Overall error ranking (lower is better)



LIMITATIONS & DFT VALIDATION CHECKLIST

Important Caveats

1. CrystalNN ≈ local neighbor estimate, not bond proof
2. 3-coord + 1.42 Å deviation = structural proxy only
3. No atom-resolved planarity or electronic structure
4. graphene_score is ranking heuristic, not probability
5. Tree ensemble is independent baseline — no claim of superiority without test metrics

DFT / Experiment Checklist

- ☐ Band structure → Dirac cone / linear dispersion?
- ☐ Phonon spectrum → dynamical stability?
- ☐ 2D cleavage energy / layer binding?
- ☐ Formation energy vs known phases
- ☐ Electronic DOS near Fermi level
- ☐ Compare predicted vs computed E_form, gap, E_hull
- ☐ If promising → experimental synthesis path

TAKEAWAYS

01

End-to-end screening pipeline

MP data → Geometric-Bias Transformer → structural proxies → ranked candidates

02

Multi-task property prediction

Formation energy, band gap, energy above hull learned jointly from structure

03

Honest ranking, not discovery claim

graphene_score prioritizes; DFT & experiment decide

04

Ready for next iteration

Top candidates + checklist → focused validation experiments

Questions?

Live Demo: aistudio.google.com/apps/94414f00-4777-400f-9449-23bd8e02f8ff

발표 시간: 10분